

Programación avanzada

Patrones de Diseño



Ing. Matias Macrino

Patrones de diseño

- ▶ Patrones creacionales:

- ▶ MVC (Model View Controller):

- > Separa los datos de la lógica de negocio de una aplicación de la interfaz de usuario y el modulo encargado de gestionar los eventos y comunicaciones.

- > Las vistas y los controladores suelen estar muy relacionados, estos últimos tratan los eventos que se producen en la UI (User Interface), ya sea grafica, por consola o web.

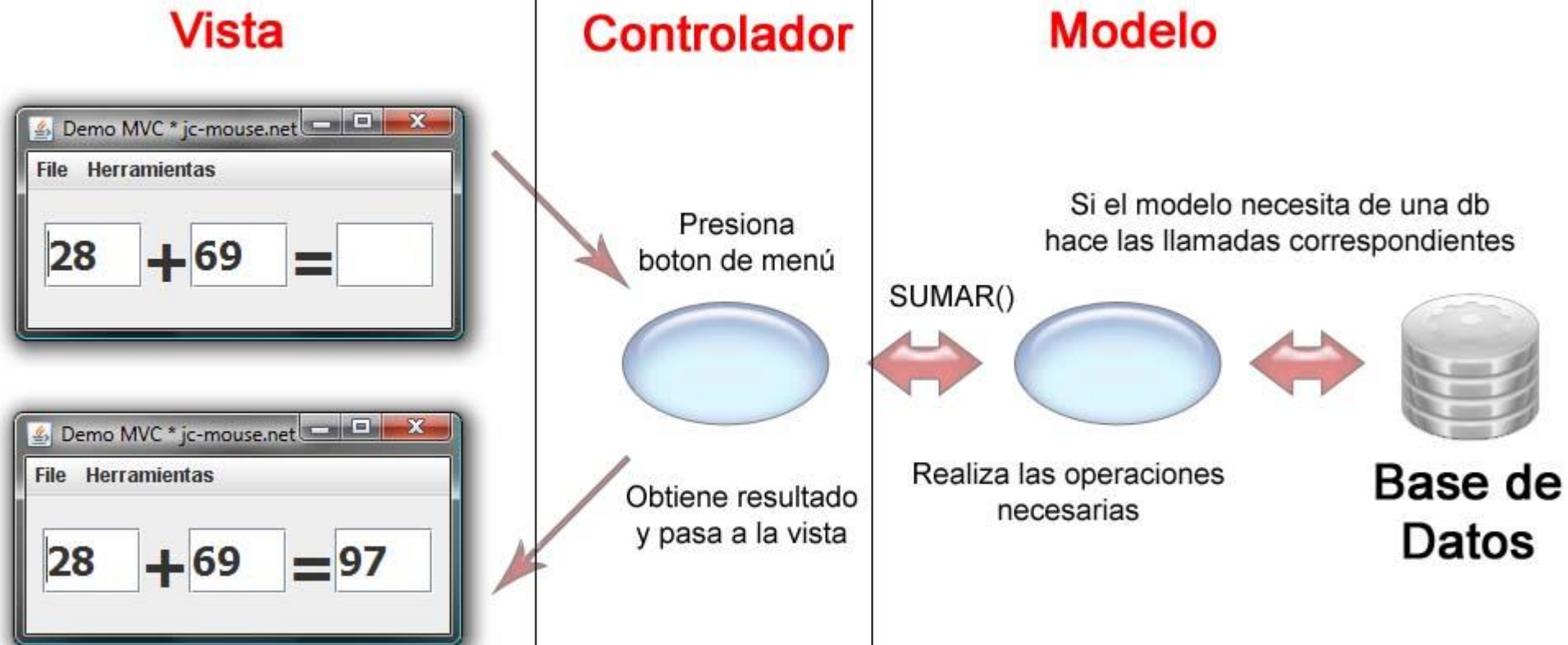
Patrones de diseño

► Patrones creacionales MVC como funciona:

Flujo de control:

1. El usuario realiza una acción en la interfaz
2. El controlador trata el evento de entrada
 - > Previamente se ha registrado
3. El controlador notifica al modelo la acción del usuario, lo que puede implicar un cambio del estado del modelo (si no es una mera consulta)
4. Se genera una nueva vista. La vista toma los datos del modelo
 - > El modelo no tiene conocimiento directo de la vista
5. La interfaz de usuario espera otra interacción del usuario, que comenzará otro nuevo ciclo

Patrones de diseño



Patrones de diseño

- ▶ Patrones creacionales:

- ▶ MVC (Model View Controller):

Model: Capa que representa lo que tiene que ver con el acceso a los datos, CRUD, lógica de negocio.

View: Tiene que ver con la representación de datos del modelo y lo que se expone al usuario

Controller: Es el puente entre la vista y el modelo, recibe eventos generados desde la vista y se encarga de direccionar al modelo dicha petición.

Patrones de diseño

- ▶ Patrones creacionales:
- ▶ MVC (Model View Controller):

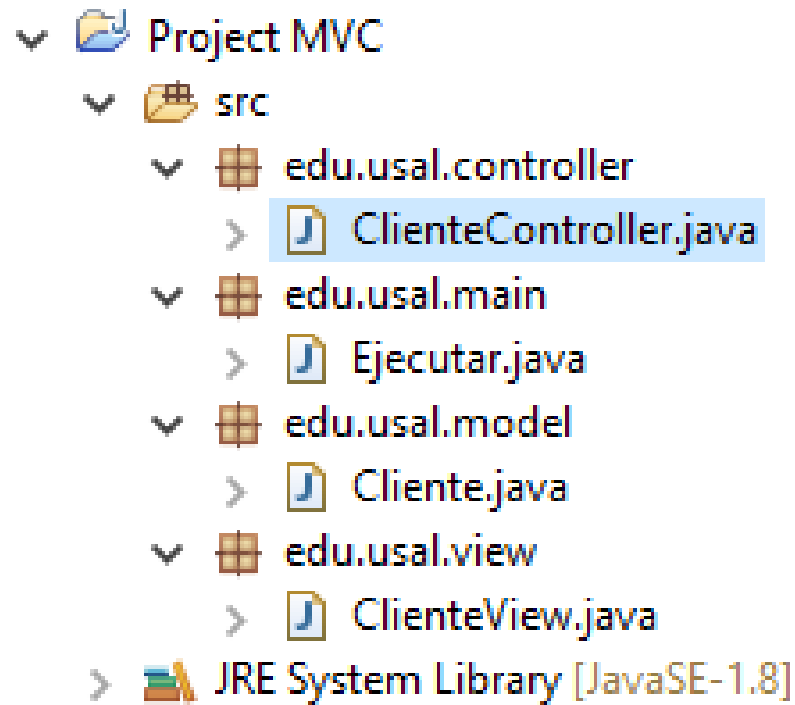
¿Qué ventaja tengo usando este patrón?

Lo importante de dicho patrón es que permite dividir en partes, que de alguna manera son independientes, con lo que por ejemplo hago algún cambio al modelo no afectaría a la vista o si hay algún cambio será muy mínimo.

Patrones de diseño

- ▶ Patrones creacionales: MVC (Model View Controller):

- ▶ Estructura del proyecto:



Patrones de diseño

- Estructura del proyecto (Model):

```
public class Cliente {  
    private int id;  
    private String nombre;  
    private String apellido;  
  
    public Cliente() { }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public String getApellido() {  
        return apellido;  
    }  
    public void setApellido(String apellido) {  
        this.apellido = apellido;  
    }  
}
```

Patrones de diseño

Estructura del proyecto (View):

```
public class ClienteView {  
    public void imprimirDatosCliente(int id,String nombre, String apellido) {  
        System.out.println("**** DATOS CLIENTE ****");  
        System.out.println("Id: "+id);  
        System.out.println("Nombre: "+nombre);  
        System.out.println("Apellido: "+apellido);  
    }  
}
```

Patrones de diseño

```
public class ClienteController {
    private ClienteView vista;
    private Cliente modelo;

    public ClienteController(Cliente modelo, ClienteView vista) {
        this.modelo = modelo;
        this.vista = vista;
    }

    public int getId() {
        return modelo.getId();
    }

    public void setId(int id) {
        this.modelo.setId(id);
    }

    public String getNombre() {
        return modelo.getNombre();
    }

    public void setNombre(String nombre) {
        this.modelo.setNombre(nombre);
    }

    public String getApellido() {
        return modelo.getApellido();
    }

    public void setApellido(String apellido) {
        this.modelo.setApellido(apellido);
    }

    public void actualizarVista() {
        vista.imprimirDatosCliente(modelo.getId(), modelo.getNombre(), modelo.getApellido());
    }
}
```

Estructura del proyecto
(Controller):

Patrones de diseño

Estructura del proyecto (Ejecución):

```
Console Problems @ Java
<terminated> Ejecutar [Java Application]
**** DATOS CLIENTE ****
Id: 1
Nombre: Elivar
Apellido: Largo
**** DATOS CLIENTE ****
Id: 1
Nombre: Luis
Apellido: Largo
```

```
public class Ejecutar {
    public static void main (String [] args){
        Cliente modelo= llenarDatosCliente();
        ClienteView vista  = new ClienteView();

        ClienteController controlador = new ClienteController(modelo, vista);

        controlador.actualizarVista();

        controlador.setNombre("Luis");
        controlador.actualizarVista();
    }

    private static Cliente llenarDatosCliente() {
        Cliente cliente = new Cliente();
        cliente.setId(1);
        cliente.setNombre("Elivar");
        cliente.setApellido("Largo");
        return cliente;
    }
}
```

PROGRAMACIÓN AVANZADA

MULTITAREA



Ing. Matias Macrino

APLICACIONES MULTITAREA

Multitarea: o también conocido como multihilo, es la capacidad de realizar tareas en paralelo o concurrentes, tanto a nivel de procesador como a nivel de sistema operativo.

Se conoce como tarea o **thread (hilo)** a un proceso individual que se ejecuta dentro del sistema. En principio, se podría considerar que cada bloque de código incluido en un método es un **thread**. El método **main()** representa, de hecho, el conocido como **thread principal**.

Normalmente, los programas son de flujo único, lo que significa que las tareas se ejecutan de una en una y la siguiente tarea no comienza hasta que la anterior haya terminado.

APLICACIONES MULTITAREA

Cuando se habla de ejecución en paralelo esto no significa que las tareas se estén ejecutando simultáneamente, ya que para llevar esto a cabo sería necesario disponer de más de un procesador.

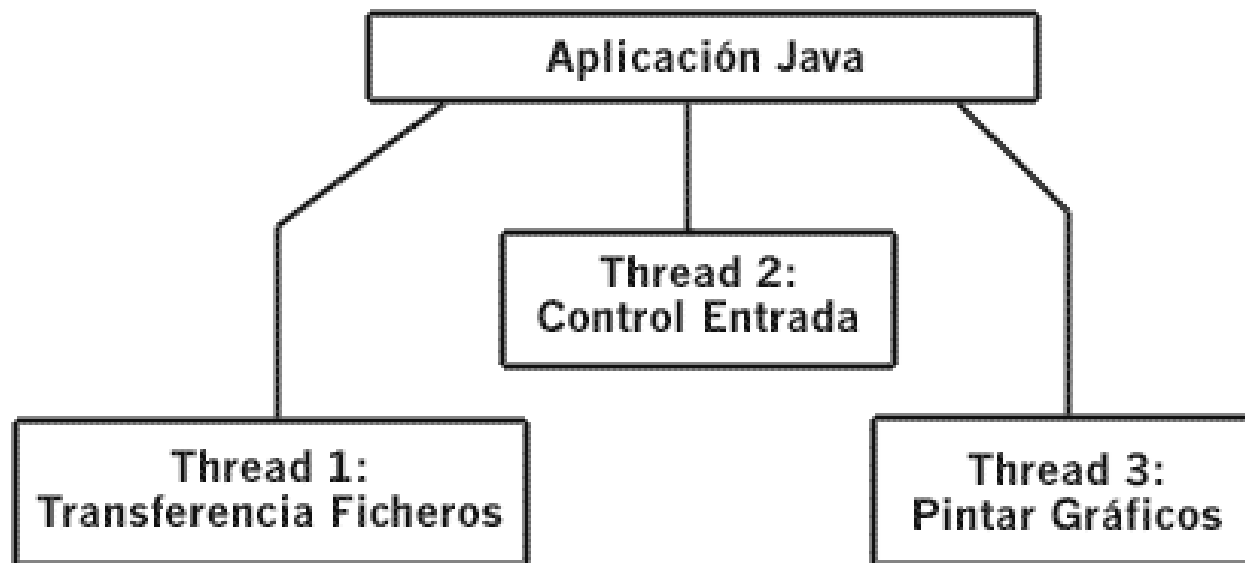
En equipos monoprocesador la ejecución en paralelo, conocida como ejecución en modo multitarea, significa que no es necesario que una tarea termine para que otra se pueda ejecutar, pudiendo la **JVM** repartir el tiempo de uso de la CPU entre las tareas que tiene que ejecutar.

Se tiene que tener en cuenta que **no se puede hacer suposición alguna sobre el tiempo que la CPU dedicará a cada tarea**, así como el orden de ejecución de estas, ya que todas estas variables dependen de la implementación de la **Maquina Virtual**.

APLICACIONES MULTITAREA

Cada **thread** controla un único aspecto dentro de un programa.

Todos los **threads** comparten los mismos recursos, al contrario que los procesos.



APLICACIONES MULTITAREA

Un **hilo** en **Java** puede estar en cualquiera de los siguientes cuatro estados:

- ❑ **Nuevo**
- ❑ **Ejecutable**
- ❑ **Bloqueado**
- ❑ **Terminado**

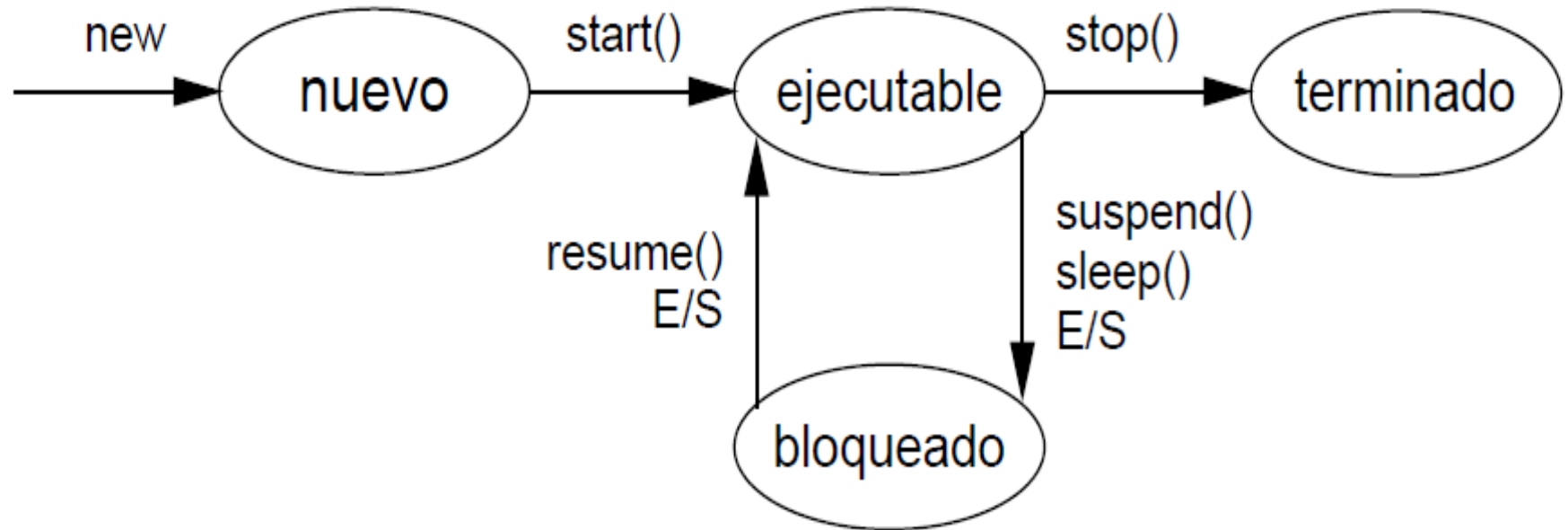


Diagrama de estados de los hilos en JVM.

APLICACIONES MULTITAREA

El paquete java.lang incluye dos elementos para la creación de aplicaciones multi thread en java:

- Clase derivada de la clase abstracta **Thread**
- Implementación de la interface **Runnable**

Extensión de la clase Thread: La creación y ejecución de tareas es modo multitarea utilizando este sistema requiere la realización de los siguientes pasos.

- 1 - Heredar la clase **Thread**.
- 2 - Sobreescritura del método **run()** de la clase **Thread**.
- 3 - Creación y ejecución de las tareas método **start()**.

APLICACIONES MULTITAREA

Método run():

Proporciona la base para la definición de las tareas.

Si las tareas a implementar van a realizar funciones diferentes, se debe de crear tantas subclases de **Thread** como tareas, codificando el método **run()** de cada una las instrucciones a ejecutar por cada tarea.

Consigna: se desea desarrollar un programa que realice dos tareas concurrentes. Imprimir por pantalla los números del 1 al 15, mientras que otra tarea será imprimir los números del -1 al -15.

APLICACIONES MULTITAREA

Método Start():

Cuando se instancia un objeto **Thread** esta pasa al estado de “**nuevo**”, pero aun no se produce la ejecución de la tarea asociada.

Para que esto suceda, habrá que invocar explícitamente al método **start()** de cada objeto **Thread**.

La llamada a dicho método provoca que el **thread** pase al estado de “**ejecutando**”. En esta situación el gestor de tareas de la **JVM** podrá, en cualquier momento, elegir la tarea para su ejecución invocando al método **run()**.

Consigna: se desea poner en ejecución las tareas desarrolladas con anterioridad.

APLICACIONES MULTITAREA

Métodos para control de threads:

↪ *Método Sleep(long tiempo):* Se trata de un método estático que al ser invocado pasa al estado “**bloqueado**” durando los milisegundos especificados en el argumento. Mientras el **thread** se encuentre en dicho estado no podrá volver a ejecución por el gestor de tareas, hasta que terminado el tiempo vuelva al estado “**ejecutable**”. El llamado a dicho método puede provocar una excepción de tipo **InterruptedException**

↪ *Métodos nombre de Thread:* Se puede establecer el nombre del Thread mediante su método **setName(String nombre)** o también se puede establecer cuando se crea usando su constructor **Thread(String nombre)**. Para obtener su nombre se hace a través del método **String:getName()**.

Consigna: a lo desarrollado anteriormente se desea agregar que las dos tareas tengan un reposo de 50 y 100 milisegundos y que se pueda establecer y obtener el nombre del Thread

APLICACIONES MULTITAREA

Métodos para control de threads:

↯ *Obtener Thread en ejecución:* Se dispone del método estático **currentThread()**.

Ejemplo: **Thread.currentThread().getName();**

↯ *Prioridad de un Thread:* Es un facto a tener en cuenta por el gestor de tareas a la hora de repartir la CPU entra las tareas. Es posible establecer explícitamente una prioridad a una tarea mediante el método **setPriority(int p)**, existe constantes definidas para la prioridad **MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY**.

↯ *Método yield():* Este método devuelve el **thread** en ejecución al estado de “ejecutable”, poniendo en ejecución otro **thread** de su misma prioridad. El objetivo de dicho método es promover la rotación de la CPU entre **threads** de igual prioridad.

APLICACIONES MULTITAREA

Métodos para control de threads:

↯ Método *join()*: Se utiliza para unir un **thread** con el final de otro, de modo que el primero quedara bloqueado en el estado de “**ejecutable**” hasta que finalice la ejecución del **thread** con el que se ha enlazado. Puede provocar un excepción **InterruptedException** al igual que **sleep()**.

↯ Otros métodos (deprecados):

stop(): Mata de forma forzada al **thread**.

suspend(): Pone en suspensión el **thread** sin marca de tiempo.

resume(): Activa el **thread** en suspensión, el cual fue realizado con el método **suspend()**.

APLICACIONES MULTITAREA

Implementación de la interfaz Runnable:

Al optar por esta opción, las acciones que se deben de llevar a cabo serán las siguientes:

- 1 – Implementar la interfaz y el método `run()` de la misma.
- 2 – Creación y ejecución de tareas.

La interfaz **Runnable** incluye un único método, **`run()`**, en cual se incluirán las acciones a realizar por la tarea, al igual de la clase **Thread**.

Consigna: Desarrollar un clase que implemente la interfaz **Runnable**. La tarea debe de recorrer los números del 1 al 15.

APLICACIONES MULTITAREA

Creación y ejecución de tareas.

Toda tarea en ejecución multitarea es un ***thread***. Lo que significa, que habrá que ***crear tantos objetos de la clase thread como tareas queramos poner en ejecución.***

Para su creación, debemos de utilizar alguno de los siguientes constructores.

Thread(Runnable obj);

Thread(Runnable obj, String nombre);

Consigna: Poner en ejecución la tarea desarrollada con anterioridad

APLICACIONES MULTITAREA

Sincronización de Threads.

↯ **Acceso concurrente a objetos:** al trabajar con aplicaciones multitarea, pueden darse situaciones en las cuales la utilización de un objeto en modo concurrente de lugar a una situación de incoherencia de datos.

Ejemplo: la clase Acceso se utiliza para llevar la cuenta del numero de veces que se ha pasado por un determinado bloque de código.

```
public class Accesos {  
    private int cont;  
    public int getValor(){  
        return this.cont;  
    }  
    public void setValor(int cont) {  
        this.cont = cont;  
    }  
}
```

APLICACIONES MULTITAREA

Sincronización de Threads.

Partiendo de la clase anterior, tendremos la siguiente tarea en la siguiente clase:

```
public class Tarea implements Runnable{
    Accesos accesos;
    public Tarea() {
        this.accesos = new Accesos();
    }
    @Override
    public void run() {
        int actual = this.accesos.getValor();
        actual++;
        try{
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        this.accesos.setValor(actual);
        System.out.println("Total de accesos: " + this.accesos.getValor());
    }
}
```

APLICACIONES MULTITAREA

Sincronización de Threads.

Ahora supongamos que se crean tres **Threads** asociados a la tarea mencionada con anterioridad.

```
public class Ejecucion {  
    public static void main(String[] args) {  
        Tarea tarea = new Tarea();  
        Thread t1 = new Thread(tarea);  
        Thread t2 = new Thread(tarea);  
        Thread t3 = new Thread(tarea);  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

APLICACIONES MULTITAREA

Sincronización de Threads.

*Sincronización y monitores: La sincronización de un bloque código consiste en impedir la entrada de un **thread** al bloque hasta que el ultimo **thread** en entrar a dicho bloque finalice la ejecución del mismo.*

Para controlar el proceso de sincronización, siguiendo la sintaxis indicada a continuación:

```
synchronized (referencia_objeto) {  
    //Bloque de código a sincronizar.  
}
```

Cuando un **thread** entra en un bloque sincronizado, adquiere un monitor, bloqueando la entrada de otros **threads** hasta que el monitor sea liberado. Esto sucede automáticamente cuando el **thread** finaliza la ejecución del bloque sincronizado.

APLICACIONES MULTITAREA

Sincronización de Threads.

```
public class Tarea implements Runnable{
    Accesos accesos;
    public Tarea() {
        this.accesos = new Accesos();
    }
    @Override
    public void run() {
        synchronized (accesos) {
            int actual = this.accesos.getValor();
            actual++;
            try{
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            this.accesos.setValor(actual);
            System.out.println("Total de accesos: " + this.accesos.getValor());
        }
    }
}
```

APLICACIONES MULTITAREA

Comunicación entre Threads.

Muchas veces no es suficiente con el empleo de bloques de códigos sincronizados para evitar el tipo de problemas de concurrencia. Por lo cual necesitamos métodos de comunicación entre los **Threads**.

Métodos de comunicación: para esto se cuenta con tres métodos en la clase **Object** que permite enviar señales de unos **threads** a otros.

wait(): Cuando un **thread** invoca este sobre un **objeto**, libera el monitor del mismo y pasa al estado “**bloqueado**”, en el cual permanecerá hasta que reciba una señal por parte del **thread** que en ese momento tenga el monitor del objeto para poder continuar la ejecución.

APLICACIONES MULTITAREA

Comunicación entre *Threads*.

wait(int milliseconds): Es una sobrecarga del método ***wait()*** en la que se puede establecer un tiempo de espera máximo. Si transcurre el tiempo y el ***thread*** no recibió la señal para salir del estado “bloqueado”, automáticamente pasa a “ejecutable”.

notify(): Al invocarlo se envía a una señal al ***thread*** que hizo la llamada a ***wait()*** sobre el objeto. A partir de este momento el ***thread*** podrá continuar su ejecución.

notifyAll(): Se utiliza para notificar a todos los ***threads*** que están en estado “bloqueado”, por el método ***wait()***.

APLICACIONES MULTITAREA

Ejercitación – Ejercicio 1:

Desarrollar una aplicación que imprima por pantalla el abecedario, por cada hilo de ejecución que se cree y además indicar desde qué hilo se imprime.
Que cada hilo espere un tiempo proporcional antes de imprimir una letra random (ejemplo thread 1 – 2 segundos, thread 2 – 4 segundos, etc).

Ejecutar la aplicación, primero mediante la clase Thread y segundo mediante la interfaz Runnable

Implementar al menos cuatro hilos.

APLICACIONES MULTITAREA

Ejercitación – Ejercicio 2:

Desarrollar una aplicación que simule una carrera de caballos. Mostrar quién llega primero a la meta.

Ejecutar la aplicación, primero mediante la clase Thread y segundo mediante la interfaz Runnable

Ejecutar al menos 5 caballos.